PulsePlayer Design Document

CST-391 Milestone #5

Carter D. Wright

04-19-2026

Table of Contents

Abstract 4
Scope 4
Application Features 4
Out of Scope 5
Assumptions 5
Constraints 5
Feature Summary 5
Revised User Stories 6
Guest 6
User 6
User (Security Edge Case) 6
Admin 6
Data Model 6
Tables 6
Important Design Decisions 6
Relationships 7
Key Constraints 7
ER Diagram 7
System / Domain Design 8
..... 8
API Design 9
User APIs 9
Track APIs 9
Admin APIs 9
UI / UX Flow 10
Navigation 10
User Flows 10
Security & RBAC 10
Authentication 10
Authorization Layers 10
Session Data 11
Admin Promotion 11
Clean Architecture 11
Feature Implementation (Next.js UI) 11

UI Handles 11
Guest Handling 11
Architecture Evidence 12
Project Structure (Layer Separation) 12
Repository Layer (Data Access) 12
Service Layer (Business Logic & RBAC) 13
API Route Layer (Controller) 15
UI Implementation & Screenshots 17
Home Page 17
Discover Page 19
Library (User View) 19
Playlist Detail View 20
Admin Dashboard 21
Modal Forms (Create / Edit) 22
References 24

Abstract

This design document provides a comprehensive overview of the **PulsePlayer music application**, a full-stack web application built using Next.js, PostgreSQL, and RESTful APIs. The system enables users to browse music, manage playlists, and interact with a dynamic music catalog through a modern user interface.

The application implements **authentication and authorization using NextAuth (Auth.js)**, supporting both OAuth and credential-based login. Role-Based Access Control (RBAC) is enforced across the system to differentiate between guest, authenticated user, and administrator behaviors.

The backend architecture follows a **Clean Architecture pattern**, separating concerns across API route handlers, service layers, and repository layers to ensure maintainability, scalability, and security. The data model is normalized and uses relational database principles, including many-to-many relationships for playlist management.

This document defines the system architecture, data model, API design, UI/UX flows, security mechanisms, and implementation details. It reflects the final implementation delivered in Milestone 5 and aligns with course requirements for full-stack application development.

REPO: https://github.com/Cademic/PulsePlayer
LIVE APP: https://cst391-music-app-psi.vercel.app/

Scope

This document covers the **entire PulsePlayer application**, including both frontend and backend systems:

Application Features

- Music browsing (albums and tracks)
- Playlist management (create, edit, delete, add/remove tracks)
- User authentication using NextAuth
- Role-based access control (guest, user, admin)
- Admin dashboard for managing playlists

It also includes:

- Frontend UI built with Next.js
- Backend API routes under /api
- PostgreSQL database and data relationships

- Clean Architecture (routes → services → repositories)
- Basic security and session handling

Out of Scope

The following features are not included in this implementation:

- Playlist sharing or social features
- Advanced recommendations
- Mobile apps

Assumptions

- Authentication and database are properly configured
- Users are identified using unique IDs

Constraints

- Some legacy API routes are not fully refactored into Clean Architecture
- UI uses sidebar navigation instead of required Bootstrap navbar (noted in design alignment)
- Middleware implementation may show deprecation warnings in newer Next.js versions

---

Feature Summary

The Playlist Management feature enables users to create, manage, and interact with playlists within the PulsePlayer application. This feature is fully integrated into a **Next.js full-stack architecture**, using both frontend UI and backend API routes.

In addition to basic CRUD functionality, the system supports:

- **Authentication via NextAuth (Auth.js)** using OAuth and credentials
- **Role-Based Access Control (RBAC)** for guest, user, and admin roles
- **External track importing** (TheAudioDB integration)
- **Clean Architecture layering** for maintainability and scalability

This ensures the feature is not just functional, but also secure, scalable, and production-ready.

Revised User Stories

Guest

- Can browse Home and Discover pages
- Can access Library but sees a **restricted view with sign-in prompt**
- Cannot access playlist details or perform mutations
- Attempting protected routes triggers **redirect or 401**

User

- Can create playlists tied to their account
- Can view only playlists where owner_user_id = session.user.id
- Can:
  - Add tracks (local or external API)
  - Remove tracks
  - Rename playlists
  - Delete owned playlists

User (Security Edge Case)

- Cannot access or modify playlists owned by other users
- API enforces:
  - **403 Forbidden** for unauthorized access
  - UI displays error state

Admin

- Can access /admin dashboard
- Can:
  - View all playlists (regardless of owner)
  - Delete any playlist
- Admin routes protected via middleware + API validation

Data Model

Tables

- users
- playlists
- playlist_tracks
- tracks
- albums

Important Design Decisions

- **UUIDs** used for playlists and users (scalable + secure)
- **Integer IDs** used for tracks/albums (efficient joins)

Relationships

- One user → many playlists
- One playlist → many tracks
- One track → many playlists (junction table)

Key Constraints

- Composite PK: (playlist_id, track_id) prevents duplicates
- ON DELETE CASCADE ensures no orphaned records
- Indexed fields improve performance for filtering

ER Diagram

- Users own playlists
- Playlists contain many tracks
- Tracks belong to albums

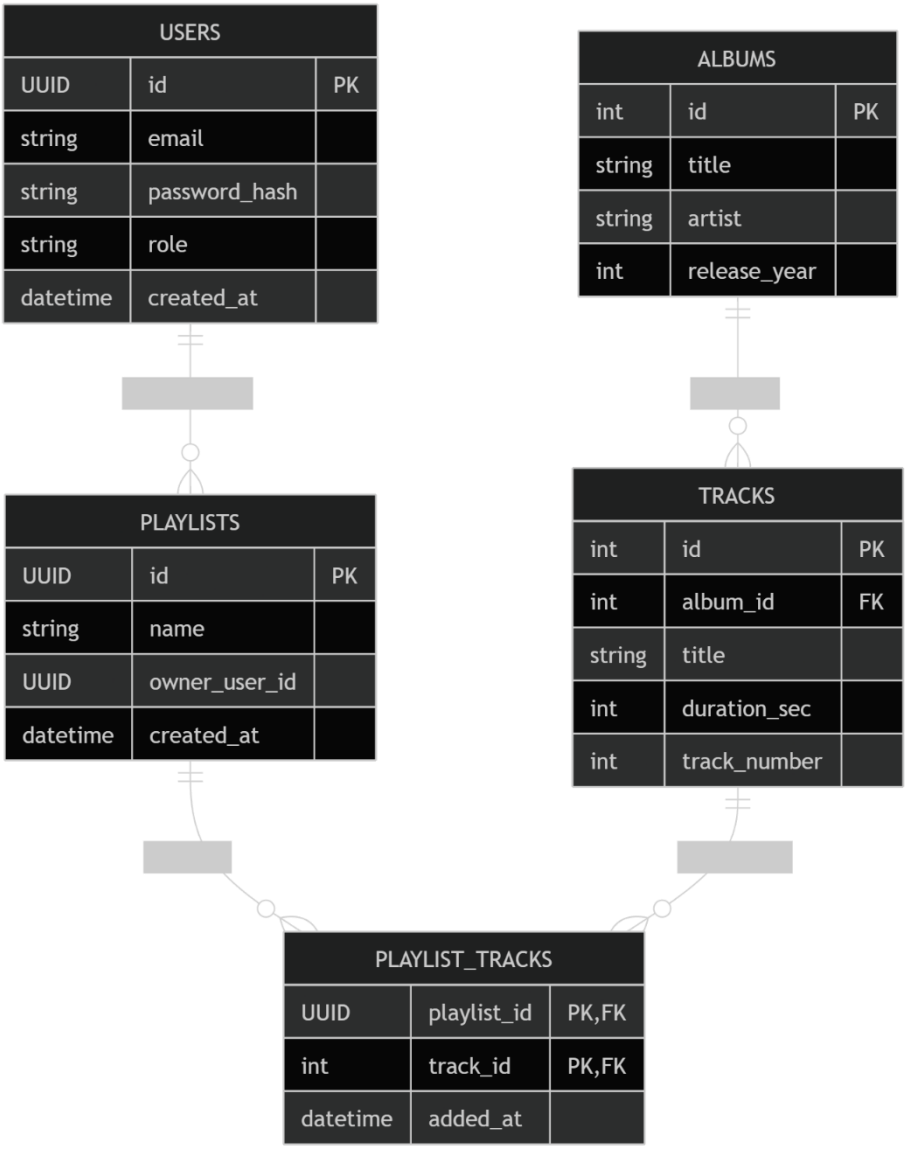


Figure 1. Entity Relationship Diagram

System / Domain Design

This UML diagram represents the conceptual structure of the PulsePlayer application. It illustrates relationships between users, playlists, and tracks at the application level. Unlike the ER diagram, this focuses on logical relationships rather than database structure.

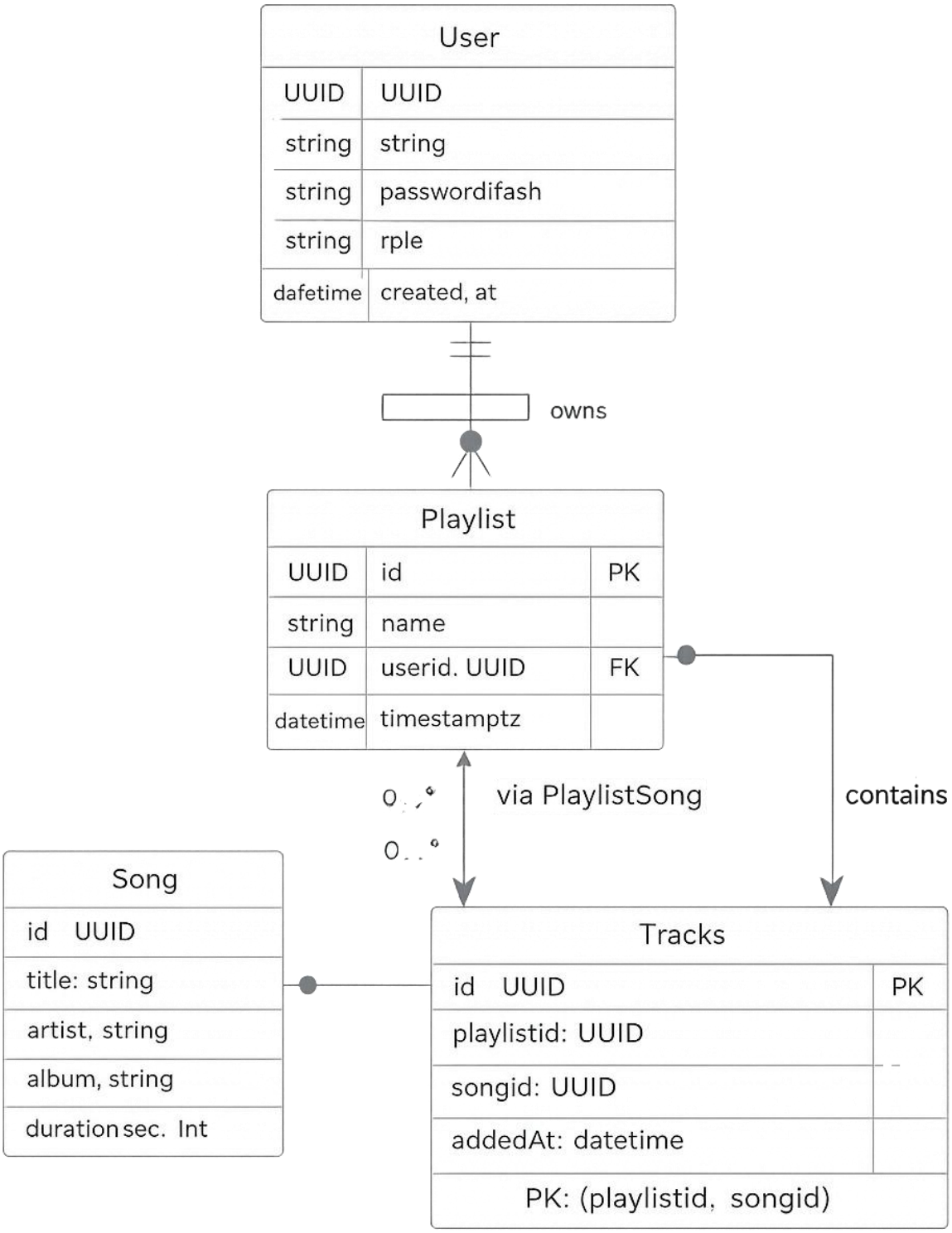


Figure 2. UML Domain Diagram

API Design

User APIs

- GET /api/playlists
  - Returns only user's playlists
  - Enforced via session filtering
- POST /api/playlists
  - Creates playlist using session user ID
  - Prevents spoofing (no manual owner input)
- GET /api/playlists/[id]
  - Checks ownership or admin role
  - Returns playlist + tracks
- PATCH /api/playlists/[id]
  - Rename playlist
  - Owner or admin only
- DELETE /api/playlists/[id]
  - Deletes playlist
  - Cascade removes tracks

Track APIs

- POST /api/playlists/[id]/tracks
  - Adds track via:
    - trackId (local)
    - audioDbTrackId (external import)
  - Prevents duplicates (409)
- DELETE /api/playlists/[id]/tracks/[trackId]
  - Removes track from playlist

Admin APIs

- GET /api/admin/playlists
  - Returns all playlists
- DELETE /api/admin/playlists/[id]
  - Deletes any playlist

UI / UX Flow

Navigation

Sidebar Items	Notes
Home	Public access
Discover	Public access
Library	Requires login for full access
Admin	Admin only

User Flows

Role	Flow
Guest	Browse → Open Library → See locked state → Sign in
User	Library → View playlists → Create → Add tracks → View details
Admin	Admin Panel → View all playlists → Delete/moderate

Security & RBAC

Authentication

- Managed by **NextAuth**
- Supports:
  - OAuth (GitHub)
  - Credentials login

Authorization Layers

1. **Middleware**
   - Protects routes (/library, /admin)
2. **UI Layer**
   - Hides unauthorized actions
3. **API Layer (Critical)**
   - Enforces all rules (source of truth)

Session Data

- user.id
- user.role

Admin Promotion

- Controlled via ADMIN_EMAILS env variable

Clean Architecture

Layer Structure

```
UI (React / Next.js)
↓
API Routes (Controllers)
↓
Services (Business Logic)
↓
Repositories (Data Access)
↓
Database (PostgreSQL)
```

Figure 3. Clean Architecture Layer Diagram

Responsibilities

- **Routes** → Handle HTTP
- **Services** → Business rules & RBAC
- **Repositories** → SQL queries

Feature Implementation (Next.js UI)

- Uses **Next.js App Router**
- Combines:
  - Server Components (data fetching)
  - Client Components (UI interaction)

UI Handles

- Loading states
- Error states (403, 404, etc.)
- Success feedback

Guest Handling

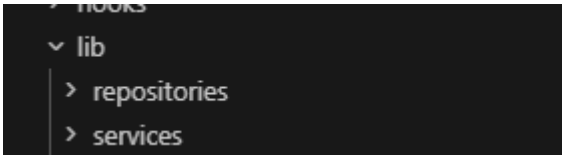
- Redirects or limited access
- Cannot perform mutations

Architecture Evidence

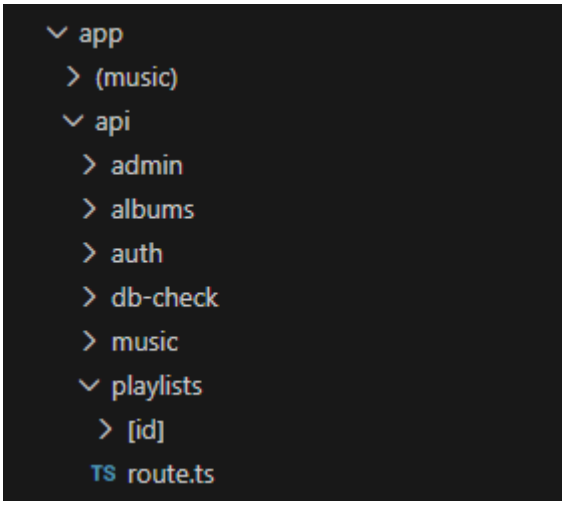
Project Structure (Layer Separation)

The following structure demonstrates a clear separation of concerns between API routes, services, and repositories, ensuring maintainability and scalability.

/lib/services & /lib/repositories



/app/api



Repository Layer (Data Access)

The repository layer is responsible for all database interactions.
As shown in the code snippets, SQL queries are isolated in this layer, ensuring that database logic is not mixed with business or API logic.

This layer is responsible for all direct database interactions, isolating SQL queries and ensuring that data access logic is separated from business logic.

/lib/repositories/playlist-repository.ts

```
export async function getPlaylistSummaryById(
  id: string
): Promise<PlaylistSummaryRow | null> {
  const result = await getPool().query<PlaylistSummaryRow>(
    `SELECT p.id, p.name, p.owner_user_id, p.created_at,
     COUNT(pt.track_id)::int AS track_count
    FROM playlists p
    LEFT JOIN playlist_tracks pt ON pt.playlist_id = p.id
    WHERE p.id = $1::uuid
```

```
      GROUP BY p.id`,
    [id]
  );
  return result.rows[0] ?? null;
}
```

```
export async function insertPlaylist(
  name: string,
  ownerUserId: string
): Promise<PlaylistSummaryRow> {
  const result = await getPool().query<
    Omit<PlaylistSummaryRow, "track_count"> & { track_count?: number }
  >(
    `INSERT INTO playlists (name, owner_user_id)
     VALUES ($1, $2::uuid)
     RETURNING id, name, owner_user_id, created_at`,
    [name, ownerUserId]
  );
  const row = result.rows[0];
  if (!row) {
    throw new Error("insertPlaylist: no row returned");
  }
  return { ...row, track_count: 0 };
}
```

```
export async function addTrackToPlaylist(
  playlistId: string,
  trackId: number,
  client?: PoolClient
): Promise<void> {
  const runner = client ?? getPool();
  await runner.query(
    `INSERT INTO playlist_tracks (playlist_id, track_id)
     VALUES ($1::uuid, $2)`,
    [playlistId, trackId]
  );
}
```

Service Layer (Business Logic & RBAC)

- Authentication checks (requireAuth)
- Authorization logic (canViewPlaylist)
- Transaction handling

The service layer handles business rules and RBAC enforcement. As shown in the code:

- User permissions are validated before actions
- Errors like **401, 403, 404** are handled here
  This ensures that security is enforced independently of the UI.

app/lib/services/playlist-service.ts

```
function canViewPlaylist(
  ownerUserId: string | null,
  userId: string,
  role: "user" | "admin"
): boolean {
  if (role === "admin") {
    return true;
  }
  if (!ownerUserId) {
    return false;
  }
  return ownerUserId === userId;
}
```

```
function requireAuth(session: Session | null): ServiceResult<{
  userId: string;
  role: "user" | "admin";
}> {
  const userId = session?.user?.id;
  const role = session?.user?.role;
  if (!userId || !role) {
    return { ok: false, error: "Unauthorized", status: 401 };
  }
  return { ok: true, data: { userId, role } };
}
```

```
export async function addTrackToPlaylist(
  session: Session | null,
  playlistId: string,
  trackId: number
): Promise<ServiceResult<{ playlistId: string; trackId: number }>> {
  const auth = requireAuth(session);
  if (!auth.ok) {
    return auth;
  }
  const summary = await playlistRepo.getPlaylistSummaryById(playlistId);
  if (!summary) {
    return { ok: false, error: "Playlist not found", status: 404 };
  }
  if (
    !canMutatePlaylist(
      summary.owner_user_id,
      auth.data.userId,
      auth.data.role
    )
```

```
  ) {
    return { ok: false, error: "Forbidden", status: 403 };
  }
  const exists = await playlistRepo.trackExists(trackId);
  if (!exists) {
    return { ok: false, error: "Track not found", status: 404 };
  }
  const client = await getPool().connect();
  try {
    await client.query("BEGIN");
    await playlistRepo.addTrackToPlaylist(playlistId, trackId, client);
    await client.query("COMMIT");
    return { ok: true, data: { playlistId, trackId } };
  } catch (err: unknown) {
    await client.query("ROLLBACK");
    const code = (err as { code?: string })?.code;
    if (code === "23505") {
      return { ok: false, error: "Track already in playlist", status: 409 };
    }
    throw err;
  } finally {
    client.release();
  }
}
```

API Route Layer (Controller)

- Calls to service layer:

```
    const result = await playlistService.listMine(session);
```

Explanation:
API routes act as controllers.
As shown in the code (page 13) :

- They handle HTTP requests
- Delegate logic to services
- Return structured responses

This keeps the routes lightweight and focused on request/response handling.

/app/api/playlists/[id]/route.ts

```
import { NextRequest, NextResponse } from "next/server";
import { auth } from "@/auth";
import * as playlistService from "@/lib/services/playlist-service";

export const runtime = "nodejs";

export async function GET() {
  try {
    const session = await auth();
    const result = await playlistService.listMine(session);
```

```
    if (!result.ok) {
      return NextResponse.json({ error: result.error }, { status:
result.status });
    }
    return NextResponse.json(result.data);
  } catch (error) {
    console.error("GET /api/playlists error:", error);
    return NextResponse.json(
      { error: "Failed to fetch playlists" },
      { status: 500 }
    );
  }
}

export async function POST(request: NextRequest) {
  try {
    const session = await auth();
    const body = await request.json();
    const name = body?.name as string | undefined;
    const result = await playlistService.createPlaylist(session, name ?? "");
    if (!result.ok) {
      return NextResponse.json({ error: result.error }, { status:
result.status });
    }
    return NextResponse.json(result.data, { status: 201 });
  } catch (error) {
    console.error("POST /api/playlists error:", error);
    return NextResponse.json(
      { error: "Failed to create playlist" },
      { status: 500 }
    );
  }
}
```

UI Implementation & Screenshots

This section provides visual evidence of the PulsePlayer user interface and demonstrates feature implementation for different roles and core functionality.

Home Page

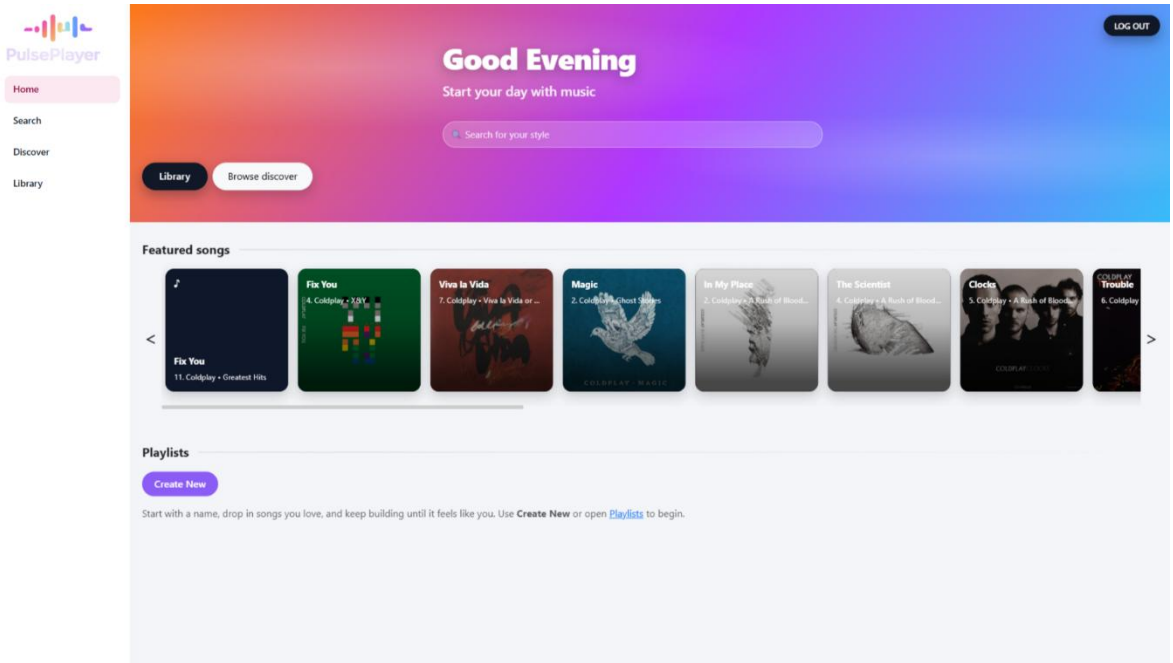


Figure 4. Home Page Interface (Guest)

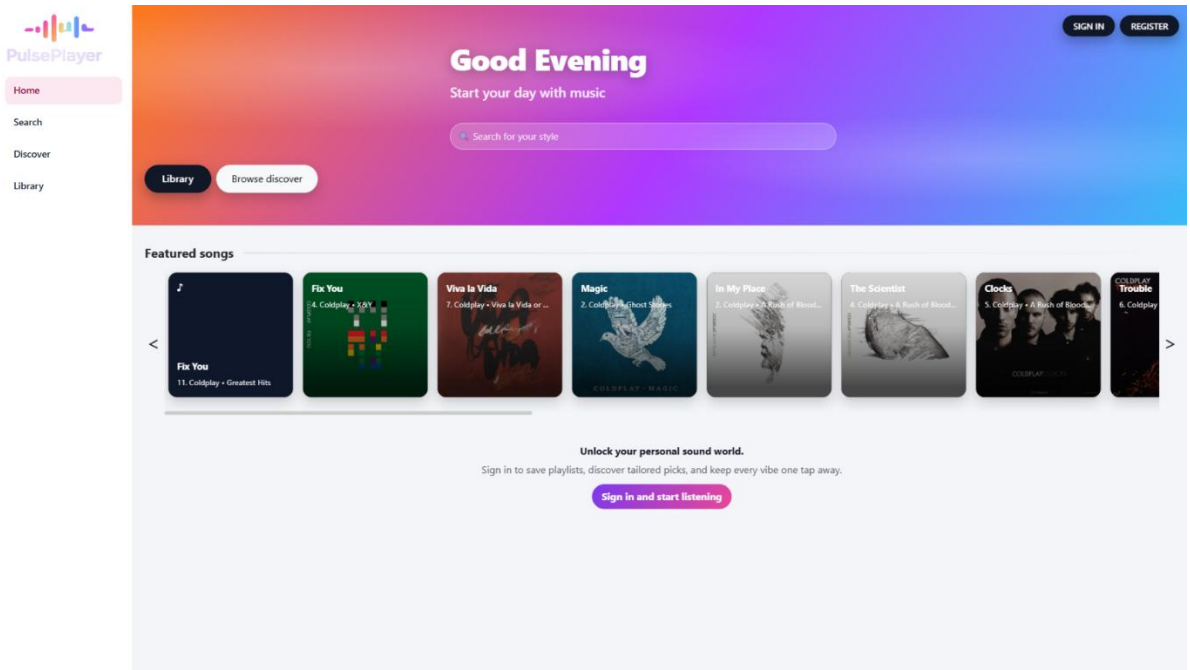


Figure 5. Home Page Interface (User)

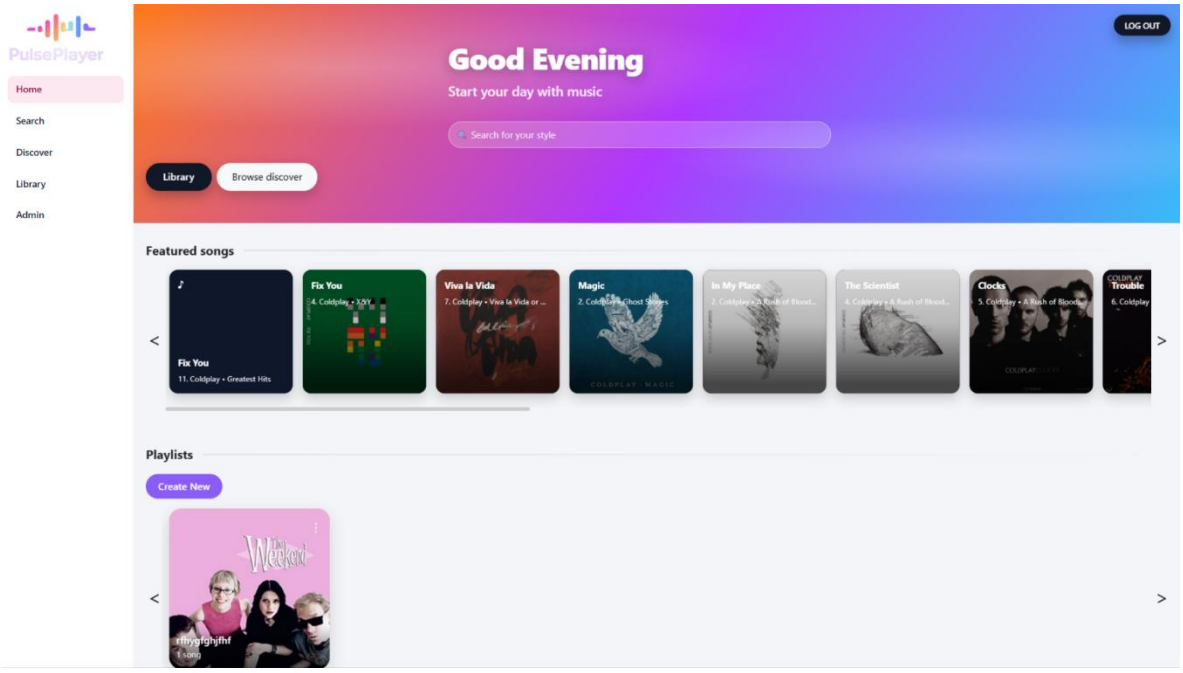


Figure 6. Home Page Interface (Admin)

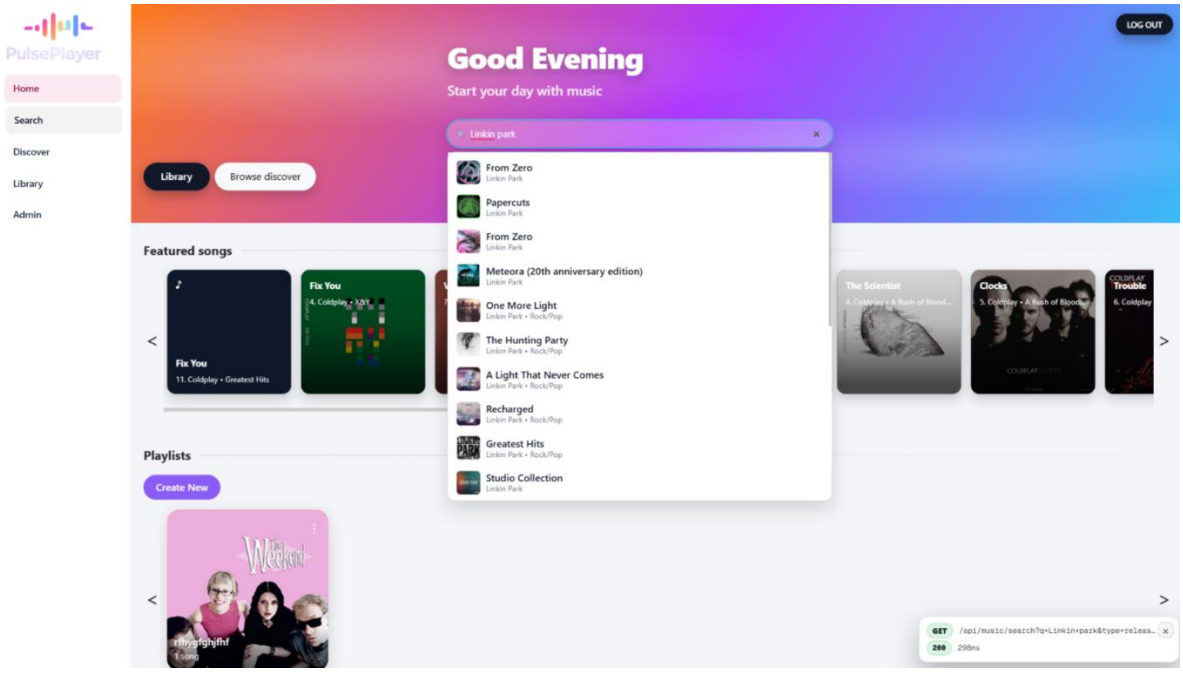


Figure 7. Home Page Interface (Search)

Description:

- Displays featured songs and playlists
- Includes global search functionality
- Serves as the main entry point for users

Discover Page

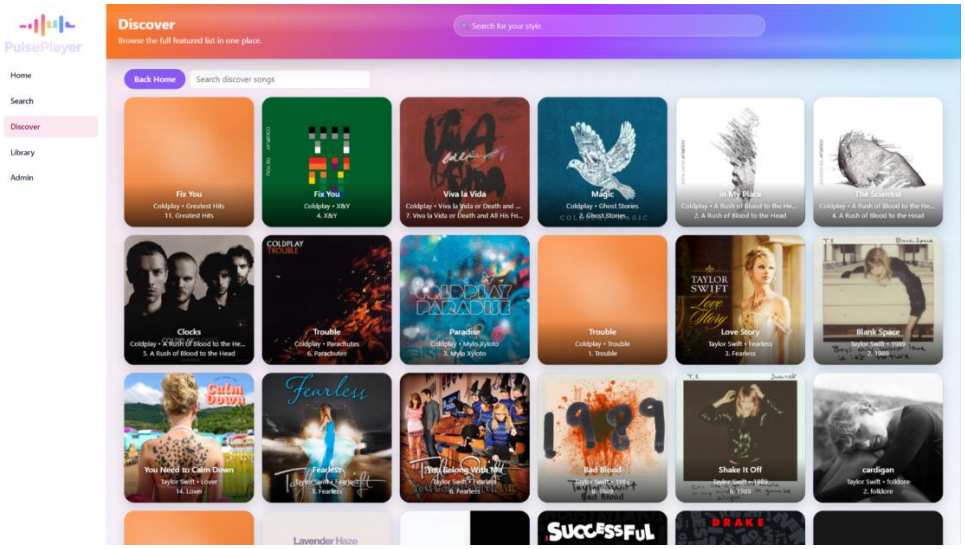


Figure 8. Discover Page Interface

Description:

- Shows a grid of albums and tracks
- Allows users to browse and discover music
- Supports search filtering

Library (User View)

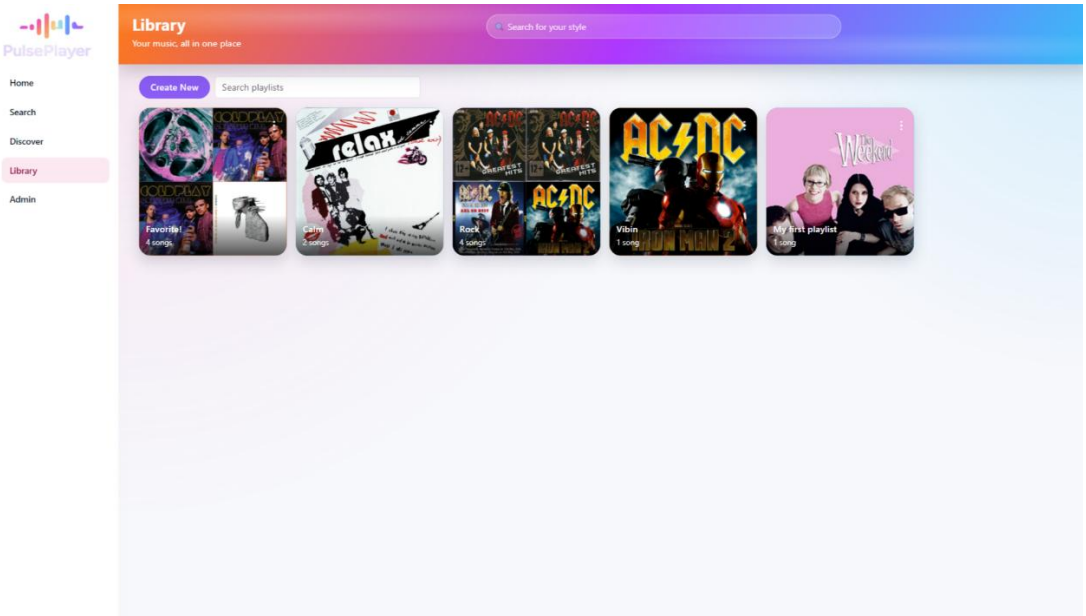


Figure 9. User Library View

Description:

- Displays user-created playlists
- Allows playlist creation and selection
- Restricted to authenticated users

Playlist Detail View

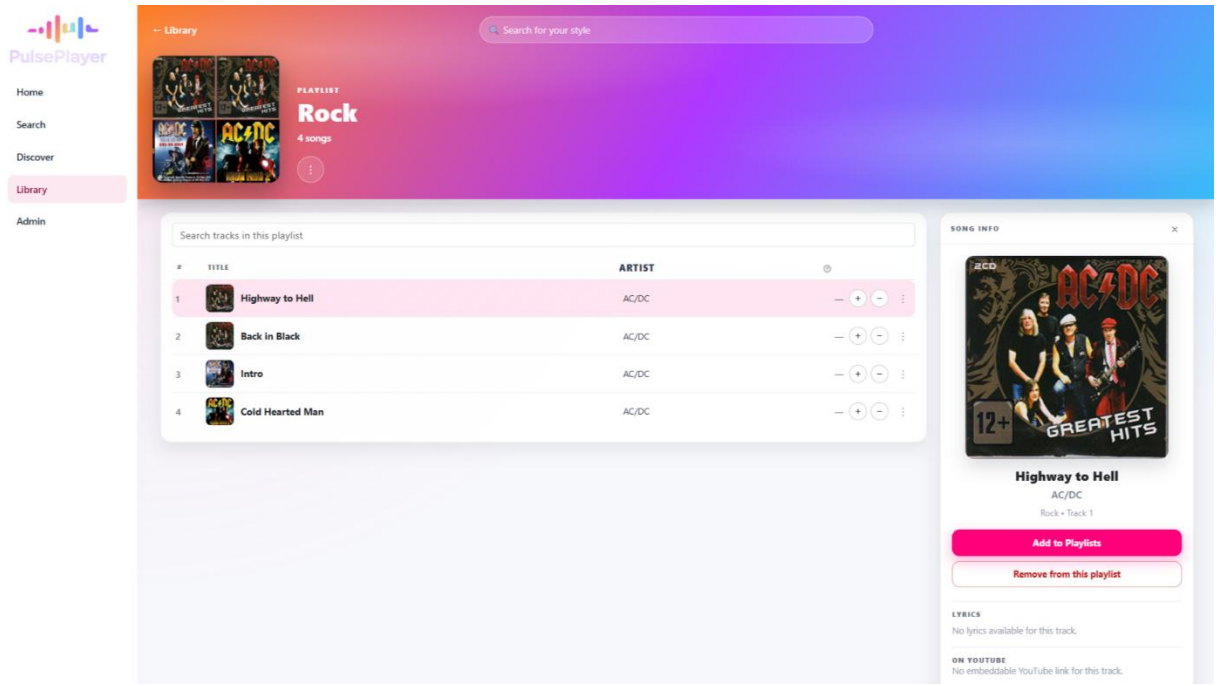


Figure 10. Playlist Detail View

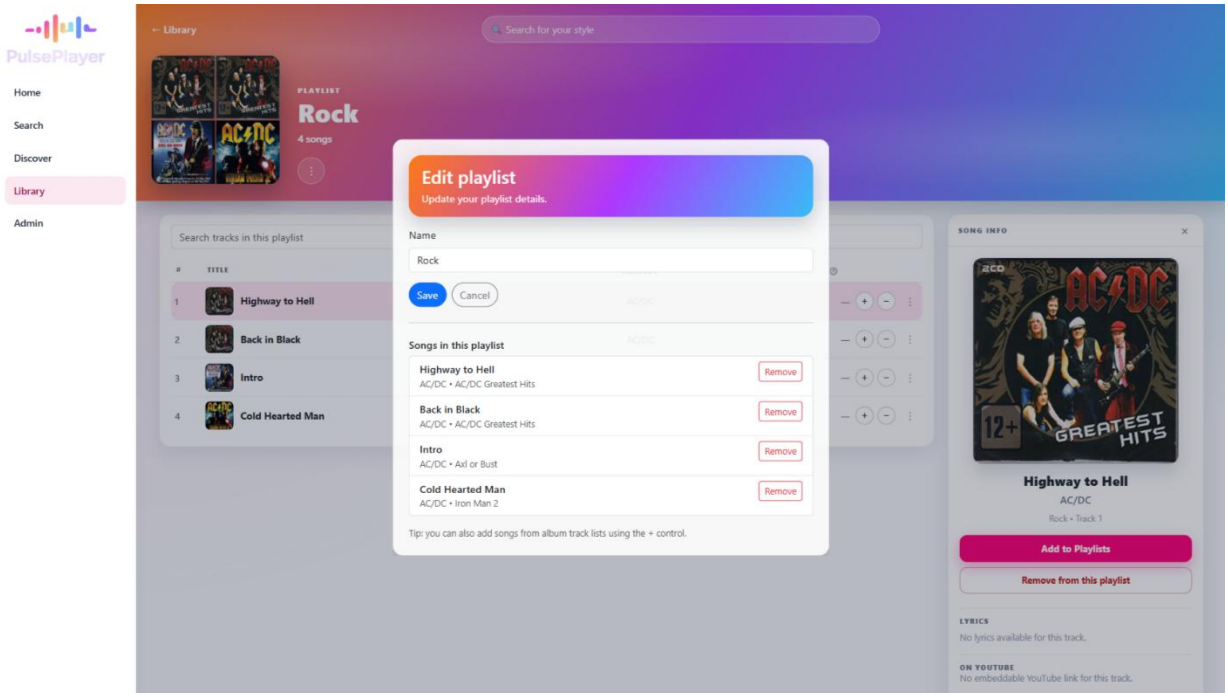


Figure 11. Playlist Detail View (edit modal)

Description:

- Displays tracks within a playlist
- Allows adding and removing tracks
- Shows song metadata and details

Admin Dashboard

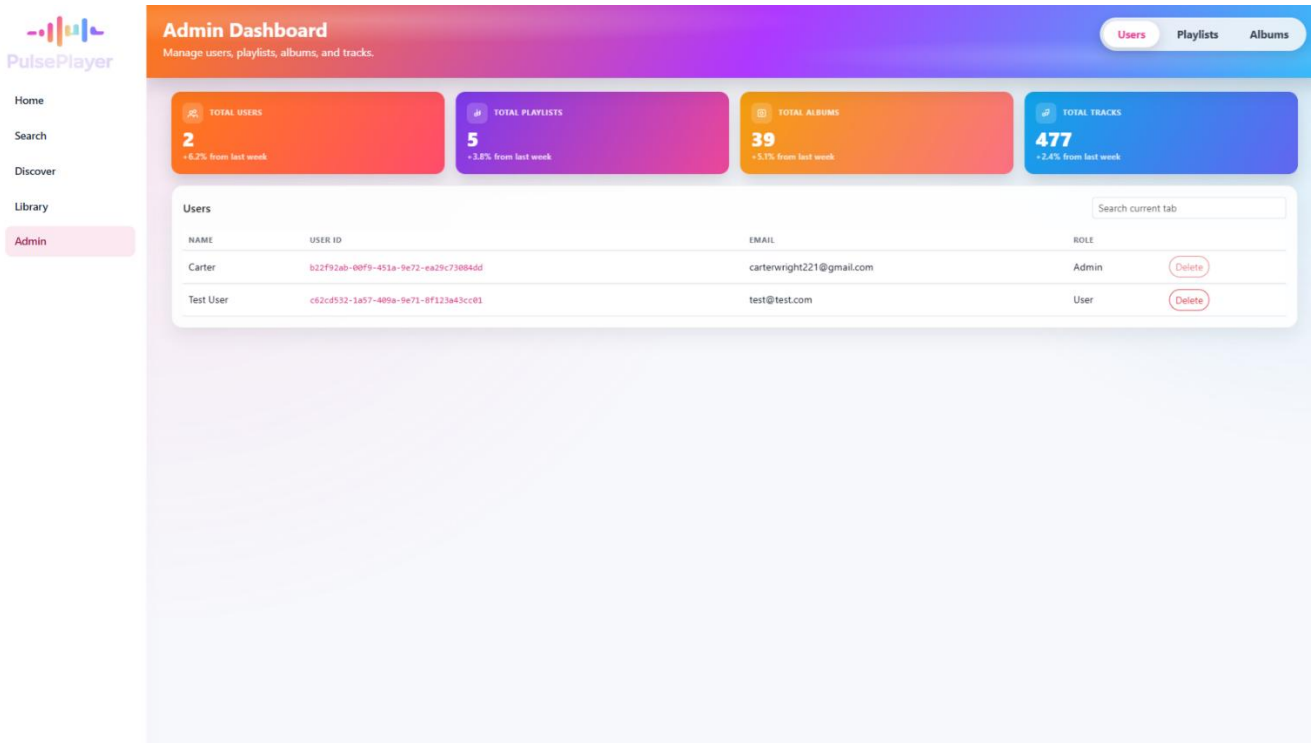


Figure 12. Admin Dashboard Interface (users)

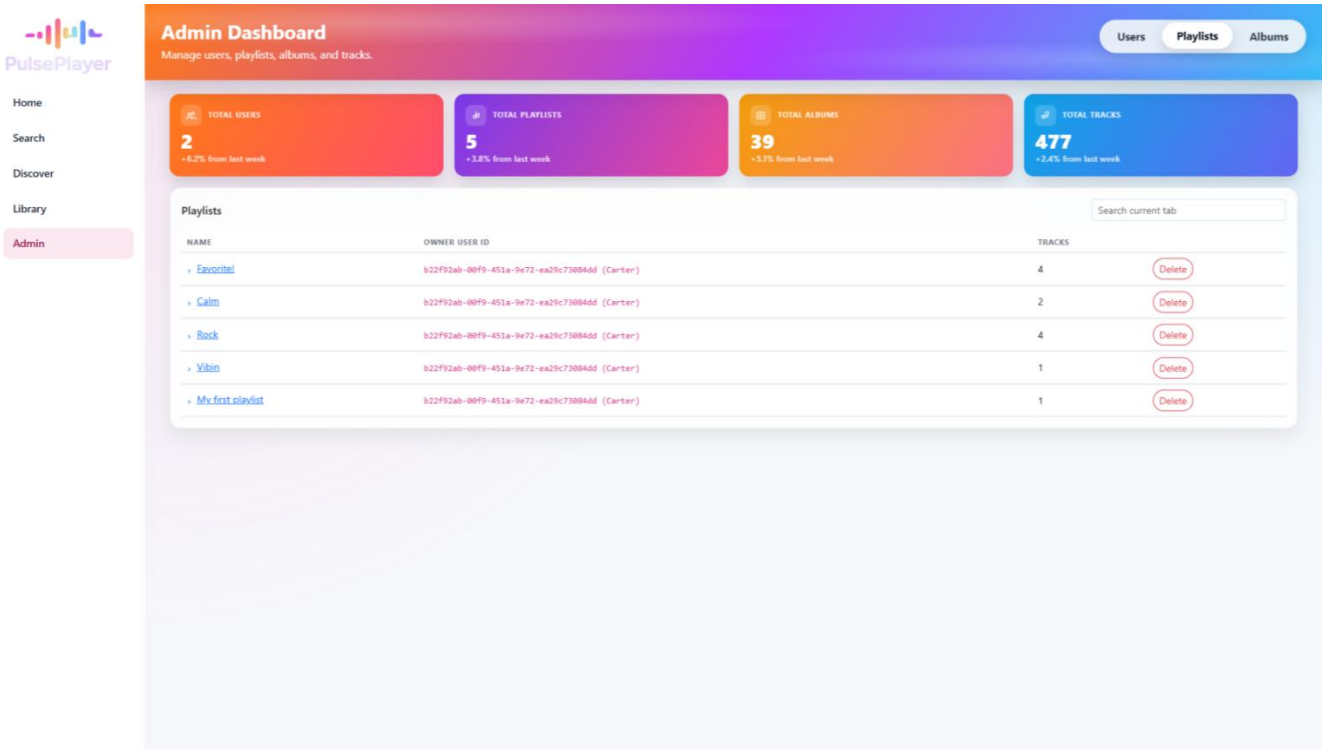


Figure 13. Admin Dashboard Interface (playlists)

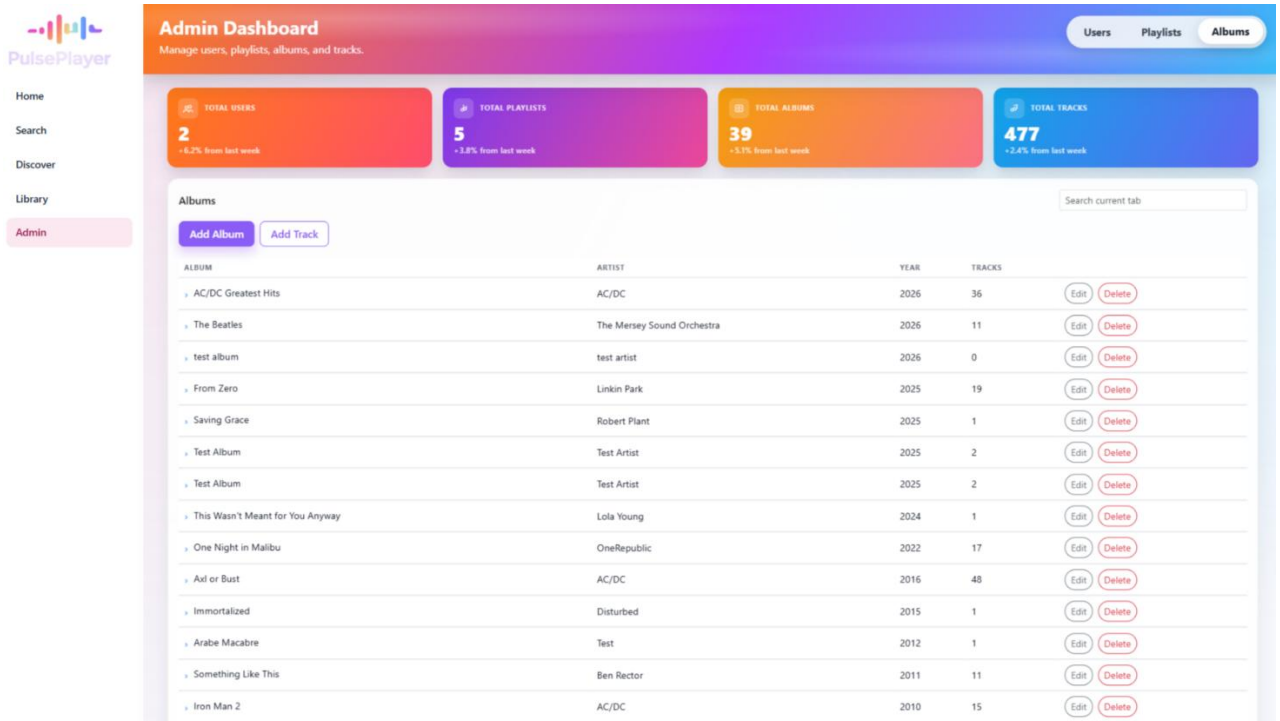


Figure 14. Admin Dashboard Interface (albums)

Description:

- Displays system statistics (users, playlists, tracks)
- Provides navigation between admin features

Modal Forms (Create / Edit)

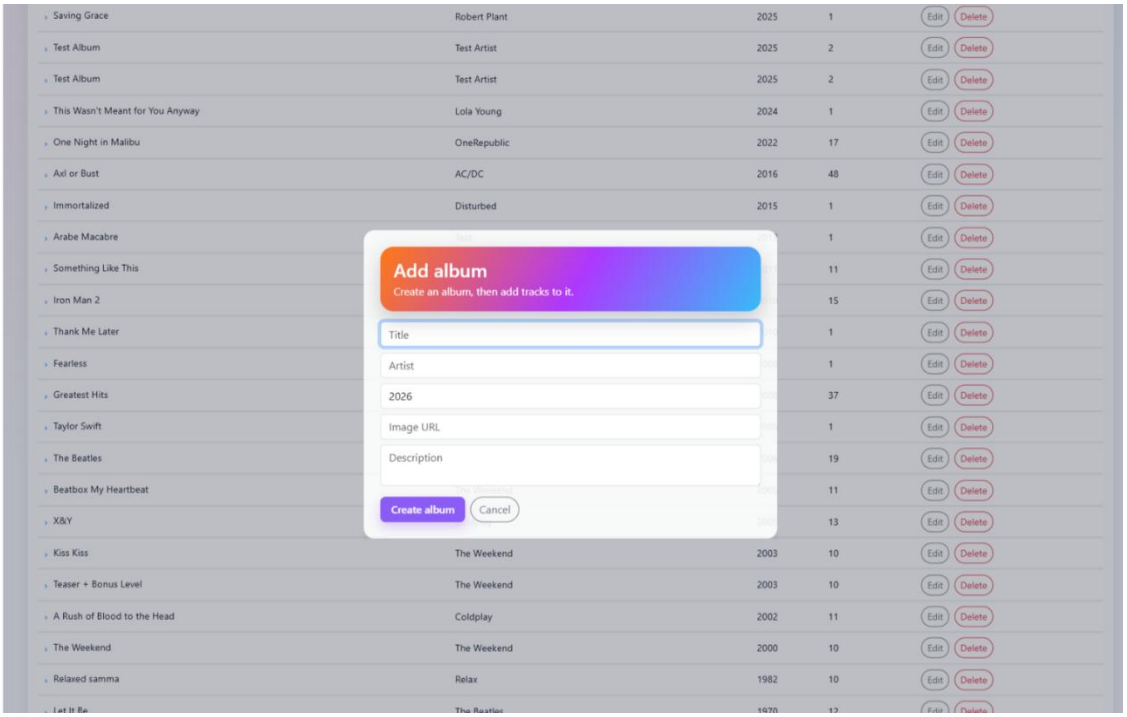


Figure 15. Modal Form for Add Album

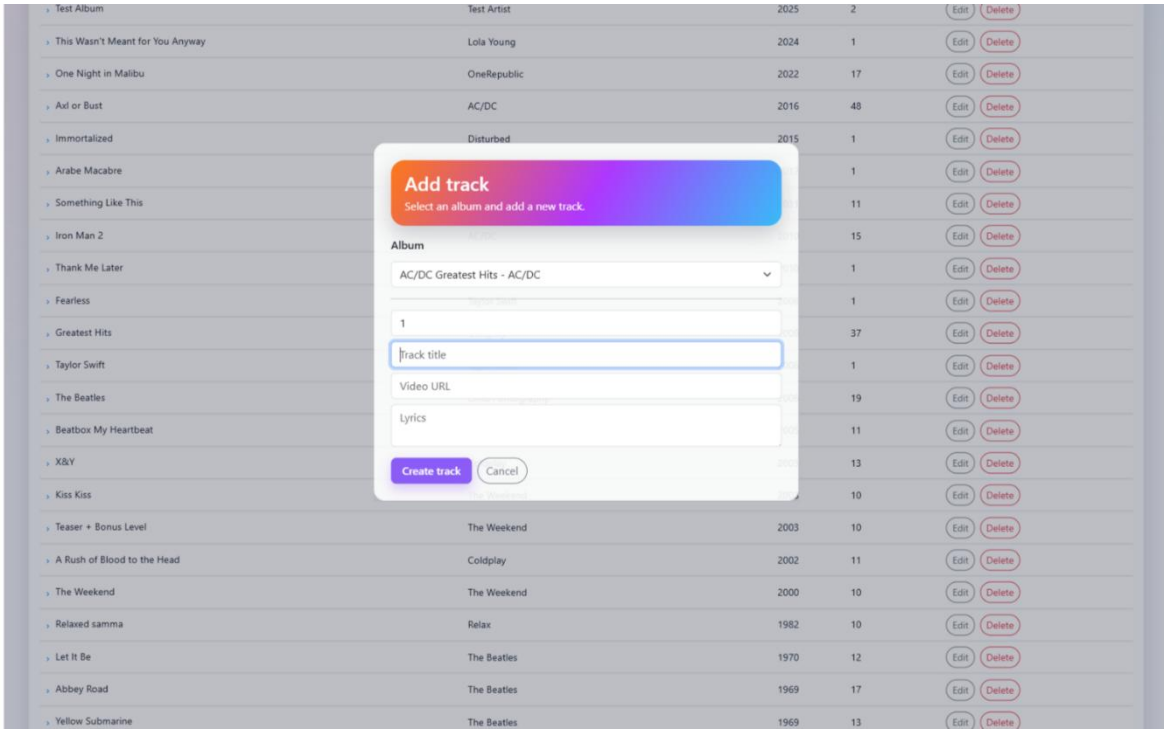


Figure 16. Modal Form for Add Track

Description:

- Used for creating albums, tracks, and playlists
- Provides validation and user feedback

References

Core Technologies

Vercel. (n.d.). *Next.js documentation*. https://nextjs.org/docs

Auth.js. (n.d.). *NextAuth.js documentation*. https://authjs.dev

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Node.js Foundation. (n.d.). *Node.js documentation*. https://nodejs.org/en/docs

Frontend / UI

React. (n.d.). *React documentation*. https://react.dev

Tailwind Labs. (n.d.). *Tailwind CSS documentation*. https://tailwindcss.com/docs

API & External Data

TheAudioDB. (n.d.). *TheAudioDB API documentation*. https://www.theaudiodb.com/free_music_api

Version Control / Deployment

GitHub. (n.d.). *GitHub documentation*. https://docs.github.com

Vercel. (n.d.). *Vercel documentation*. https://vercel.com/docs

Project Repository

Wright, C. (2026). *PulsePlayer* [Source code]. GitHub. https://github.com/Cademic/PulsePlayer

AI Usage

OpenAI. (2026). *ChatGPT (GPT-5.3)* [Large language model]. https://chat.openai.com